

Introducción

En este reporte se estará trabajando con un conjunto de datos que ya ha sido previamente utilizado y explicado en diferentes reportes anteriores. El conjunto de datos trata sobre diferentes datos recopilados mediante encuestas y generados con la técnica SMOTE sobre obesidad. Entre los datos que proporciona el dataset, se encuentran la altura, la edad, los diferentes hábitos de las personas, y su nivel de obesidad. Como se realizó previamente, el conjunto de datos fue tratado para obtener los hábitos que generaban más impacto en el nivel de obesidad, y en este reporte se estará realizando lo mismo. Se estarán clasificando a las personas en dos clases: TieneSobrepeso y NoTieneSobrepeso. Para esto estaremos observando cómo influyen los hábitos en la clasificación de estas dos clases mediante dos modelos diferentes: Linear Discriminant Analysis (LDA) y Árboles de Decisión. Con respecto a la limpieza y el tratado de datos, se eliminarán dos columnas originales en el dataset que no tendremos en cuenta en este reporte, la altura y el peso, ya que el nivel de obesidad se calcula con estas dos variables y serían altamente influyentes comparado con los hábitos, que es donde queremos la atención en el reporte. Aparte, la información recopilada sobre los hábitos será pasada a números mediante técnicas de codificación como One-Hot Encoding, Ordinal Encoding y Label Encoding. Se clasificarán en las dos clases mediante el nivel de obesidad de cada persona; en el estudio se dividen como "Insufficient Weight", "Normal Weight", "Overweight I y II" y "Obesity type I, II, III". Se determinará que una persona tiene sobrepeso si tiene tanto Overweight de cualquier tipo como Obesity de cualquier tipo. Esto hace que el balance de clases esté un poco desproporcionado al contar con más casos que se asignarán a la clase TieneSobrepeso. Esta clase se contará como una salida positiva (1) y contiene el 73% de los registros del dataset, siendo el 27% restante los casos que se consideran como NoTieneSobrepeso. Se estará manteniendo este balance de clases en las fases de train y test para que el modelo aprenda realmente sobre qué hábitos influyen en el sobrepeso y no se confunda a la hora de tener diferentes distribuciones. Esta preservación del balance es fundamental para realizar una evaluación justa del modelo, asegurando que las métricas de desempeño reflejen su capacidad real de generalización ante la proporción real de las clases. Con respecto al train y test, los datos se dividirán con un 80% de train y un 20% de test para observar la eficacia del modelo.

```
In [3]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```

df = pd.read_csv('Datos.csv')
print(df.shape)
df.head()

# Eliminar Height y Weight para evitar sesgo por IMC
df = df.drop(columns=['Height', 'Weight'])

# Binarizar variable de salida
obeso_labels = [
    'Obesity_Type_I', 'Obesity_Type_II', 'Obesity_Type_III',
    'Overweight_Level_I', 'Overweight_Level_II'
]
df['TieneSobrepeso'] = df['Obesity'].apply(lambda x: 1 if x in obeso_labels else 0)

# Balance de clases
conteo = df['TieneSobrepeso'].value_counts()
proporcion = df['TieneSobrepeso'].value_counts(normalize=True) * 100

balance = pd.DataFrame({
    'Clase': ['NoTieneSobrepeso (0)', 'TieneSobrepeso (1)'],
    'Conteo': [conteo[0], conteo[1]],
    'Porcentaje': [proporcion[0], proporcion[1]]
})

print('Balance de clases en la variable de salida ExcesoDePeso:')
print(balance.to_string(index=False))

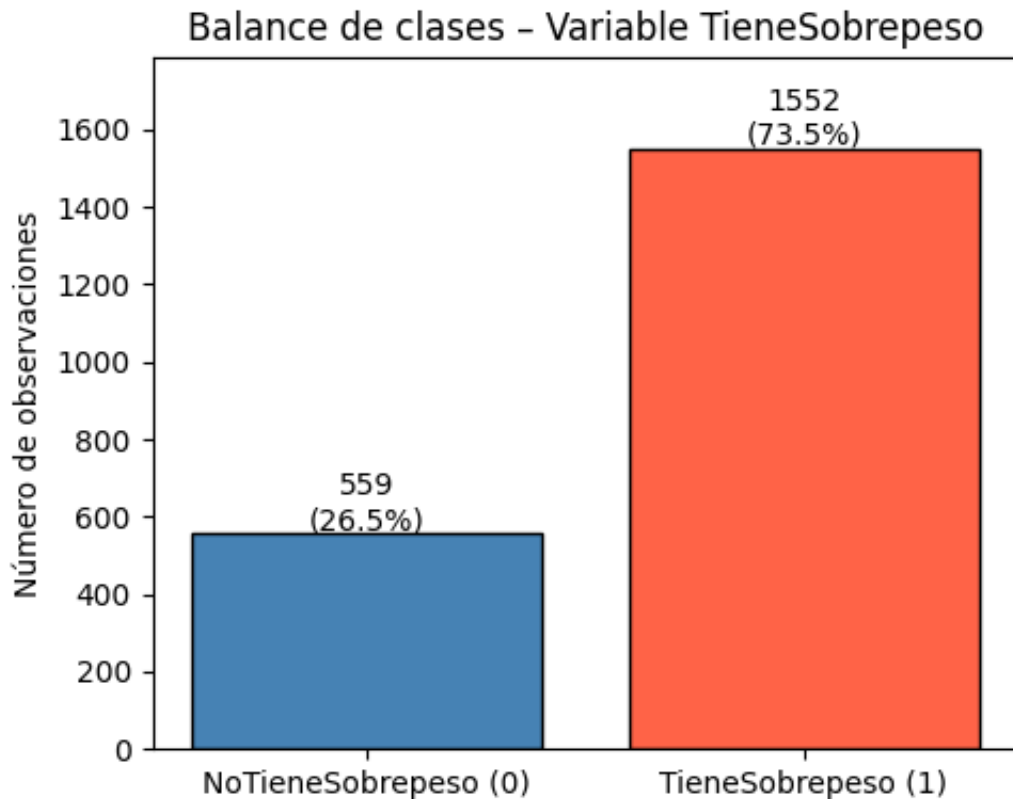
fig, ax = plt.subplots(figsize=(5, 4))
ax.bar(['NoTieneSobrepeso (0)', 'TieneSobrepeso (1)'], [conteo[0], conteo[1]],
       color=['steelblue', 'tomato'], edgecolor='black')
ax.set_ylabel('Número de observaciones')
ax.set_title('Balance de clases – Variable TieneSobrepeso')
for i, v in enumerate([conteo[0], conteo[1]]):
    ax.text(i, v + 10, f'{v}\n({proporcion[i]:.1f}%)', ha='center', fontweight='bold')
ax.set_ylim(0, max(conteo[0], conteo[1]) * 1.15)
plt.tight_layout()
plt.show()

```

(2111, 17)

Balance de clases en la variable de salida ExcesoDePeso:

	Clase	Conteo	Porcentaje
NoTieneSobrepeso (0)		559	26.480341
TieneSobrepeso (1)		1552	73.519659



Modelo basado en LDA

```
In [21]: # =====
# A2.2 – Modelo LDA
# Dataset: Obesidad (hábitos)
# Variable objetivo: TieneSobrepeso (1) / NoTieneSobrepeso (0)
# Basado en: C2_3_Linear_Discriminant_Analysis – Dr. Antonio Martínez
# =====

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OrdinalEncoder, LabelEncoder
from sklearn.metrics import (accuracy_score, confusion_matrix,
                             ConfusionMatrixDisplay, roc_auc_score, r

import warnings
warnings.filterwarnings('ignore')

# -----
# 1. CARGA Y PREPROCESAMIENTO
# -----

df = pd.read_csv('Datos.csv')
```

```

# Variable objetivo binaria
# TieneSobrepeso = 1 → Overweight o Obesity (cualquier tipo)
# NoTieneSobrepeso = 0 → Normal Weight o Insufficient Weight
sobrepeso_clases = ['Overweight_Level_I', 'Overweight_Level_II',
                    'Obesity_Type_I', 'Obesity_Type_II', 'Obesity_Type_III']
df['TieneSobrepeso'] = df['Obesity'].isin(sobrepeso_clases).astype(int)

# Eliminar Height, Weight y etiqueta original
# Razón: Height y Weight calculan directamente el IMC → data leakage
# El análisis busca entender el impacto de los Hábitos
df = df.drop(columns=['Height', 'Weight', 'Obesity'])

# Ordinal Encoding: variables con orden lógico natural
caec_order = ['no', 'Sometimes', 'Frequently', 'Always']
calc_order = ['no', 'Sometimes', 'Frequently', 'Always']
mtrans_order = ['Walking', 'Bike', 'Motorbike', 'Public_Transportation']

oe = OrdinalEncoder(categories=[caec_order, calc_order, mtrans_order],
                    handle_unknown='use_encoded_value', unknown_value=-1)
df[['CAEC', 'CALC', 'MTRANS']] = oe.fit_transform(df[['CAEC', 'CALC', 'MTRANS']])

# Label Encoding: variables binarias (yes/no → 1/0)
le = LabelEncoder()
for col in ['Gender', 'family_history', 'FAVC', 'SMOKE', 'SCC']:
    df[col] = le.fit_transform(df[col])

feature_cols = [c for c in df.columns if c != 'TieneSobrepeso']
X = df[feature_cols]
y = df['TieneSobrepeso']

print("Distribución de clases:")
print(y.value_counts())
print(f"\nProporción TieneSobrepeso : {y.mean()*100:.1f}%")
print(f"\nProporción NoTieneSobrepeso: {(1-y.mean())*100:.1f}%")

# -----
# 2. PARTICIÓN TRAIN / TEST (80/20, estratificado)
# -----

# stratify=y garantiza que el balance 73/27 se conserve en train y test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print(f"\nTrain: {len(X_train)} | Test: {len(X_test)}")
print(f"Balance train: {y_train.mean()*100:.1f}% positivos")
print(f"Balance test : {y_test.mean()*100:.1f}% positivos")

colores = {0: '#457B9D', 1: '#E63946'}
labels = {0: 'NoTieneSobrepeso', 1: 'TieneSobrepeso'}

# -----

```

```

# 3. AJUSTE DEL MODELO LDA
#
# Con 2 clases → K-1 = 1 función discriminante (LD1)
# LDA busca la proyección que maximiza la separación entre clases
# respecto a la varianza dentro de cada clase (criterio de Fisher)
# -----

lda = LinearDiscriminantAnalysis()
lda.fit(X_train, y_train)

# Proyectar datos sobre LD1
X_train_proj = lda.transform(X_train).ravel()
X_test_proj = lda.transform(X_test).ravel()

# Coeficientes de LD1 – indican el peso de cada variable en la función
coef_series = pd.Series(lda.scalings_.ravel(), index=feature_cols)
coef_df = coef_series.abs().sort_values(ascending=False)

print("\n--- Coeficientes LD1 (ordenados por magnitud) ---")
for var in coef_df.index:
    print(f" {var:15s}: {coef_series[var]:+.4f}")

# -----
# 5. PREDICCIONES Y MÉTRICAS
# -----

y_pred = lda.predict(X_test)
y_prob = lda.predict_proba(X_test)[:, 1]

cm = confusion_matrix(y_test, y_pred)
tn, fp, fn, tp = cm.ravel()
sensibilidad = tp / (tp + fn)
especificidad = tn / (tn + fp)
auc = roc_auc_score(y_test, y_prob)

print("\n" + "="*50)
print("MÉTRICAS LDA – Conjunto de Prueba")
print("="*50)
print(f" Accuracy      : {accuracy_score(y_test, y_pred)*100:.2f}%")
print(f" Sensibilidad   : {sensibilidad*100:.2f}%")
print(f" Especificidad  : {especificidad*100:.2f}%")
print(f" AUC-ROC        : {auc:.4f}")

# -----
# 6. VISUALIZACIÓN PRINCIPAL LDA
# Plot 1: Distribución de LD1 por clase + frontera lineal
# Plot 2: Coeficientes de LD1
# Plot 3: Matriz de confusión
# -----

fig, axes = plt.subplots(1, 3, figsize=(18, 5))
fig.patch.set_facecolor('#0F1117')

```

```

# — Plot 1: Distribuciones KDE sobre LD1 —————
ax1 = axes[0]
ax1.set_facecolor('#1A1D27')
for cls in [0, 1]:
    vals = X_train_proj[y_train.values == cls]
    kde = gaussian_kde(vals, bw_method=0.3)
    xg = np.linspace(X_train_proj.min() - 1, X_train_proj.max() + 1,
ax1.fill_between(xg, kde(xg), alpha=0.3, color=colores[cls])
ax1.plot(xg, kde(xg), color=colores[cls], lw=2, label=labels[cls])

ax1.axvline(0, color='white', linestyle='--', lw=1.5, label='Frontera')
ax1.set_title('Distribución en LD1 por Clase', color='white', fontsize=12)
ax1.set_xlabel('LD1', color='white')
ax1.set_ylabel('Densidad', color='white')
ax1.tick_params(colors='white')
ax1.spines[:].set_color('#444')
ax1.legend(facecolor='#1A1D27', edgecolor='#555', labelcolor='white',

# — Plot 2: Coeficientes LD1 —————
ax2 = axes[1]
ax2.set_facecolor('#1A1D27')
coefs = lda.scalings_.ravel()
sorted_idx = np.argsort(np.abs(coefs))
bar_colors = ['#E63946' if coefs[i] > 0 else '#457B9D' for i in sorted_idx]
ax2.barh([feature_cols[i] for i in sorted_idx],
         [coefs[i] for i in sorted_idx],
         color=bar_colors, edgecolor='none', alpha=0.85)
ax2.axvline(0, color='white', lw=0.8)
ax2.set_title('Coeficientes LD1\n(rojo → Sobre peso | azul → No Sobre peso)',
              color='white', fontsize=10)
ax2.set_xlabel('Peso en LD1', color='white')
ax2.tick_params(colors='white', labelsz=8)
ax2.spines[:].set_color('#444')

# — Plot 3: Matriz de confusión —————
ax3 = axes[2]
ax3.set_facecolor('#1A1D27')
disp = ConfusionMatrixDisplay(cm, display_labels=['NoSobrepeso', 'Sobrepeso'])
disp.plot(ax=ax3, colorbar=False, cmap='Blues')
ax3.set_title('Matriz de Confusión – LDA', color='white', fontsize=12)
ax3.tick_params(colors='white')
ax3.xaxis.label.set_color('white')
ax3.yaxis.label.set_color('white')
ax3.set_facecolor('#1A1D27')

fig.suptitle('LDA – Clasificación TieneSobrepeso vs NoTieneSobrepeso',
             color='white', fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()

# —————

```

```
# 7. CURVA ROC
# -----

fpr, tpr, _ = roc_curve(y_test, y_prob)

fig, ax = plt.subplots(figsize=(7, 6))
fig.patch.set_facecolor('#0F1117')
ax.set_facecolor('#1A1D27')

ax.plot(fpr, tpr, color='#457B9D', lw=2.5, label=f'LDA (AUC = {auc:.4f}')
ax.fill_between(fpr, tpr, alpha=0.08, color='#457B9D')
ax.plot([0, 1], [0, 1], '--', color='#444', lw=1.2, label='Clasificado')

ax.set_xlabel('Tasa de Falsos Positivos (1 - Especificidad)', color='w')
ax.set_ylabel('Tasa de Verdaderos Positivos (Sensibilidad)', color='w')
ax.set_title('Curva ROC - LDA', color='white', fontsize=12, fontweight='bold')
ax.tick_params(colors='white')
ax.spines[:].set_color('#444')
ax.legend(facecolor='#1A1D27', edgecolor='#555', labelcolor='white', fontcolor='white')

plt.tight_layout()
plt.show()
```

Distribución de clases:

TieneSobrepeso

1 1552

0 559

Name: count, dtype: int64

Proporción TieneSobrepeso : 73.5%

Proporción NoTieneSobrepeso: 26.5%

Train: 1688 | Test: 423

Balance train: 73.5% positivos

Balance test : 73.5% positivos

--- Coeficientes LD1 (ordenados por magnitud) ---

family_history : +1.9572

CAEC : -1.3597

SMOKE : -0.4735

CALC : +0.4015

FAVC : +0.3311

SCC : +0.2495

NCP : -0.2328

FAF : -0.2199

Gender : +0.1690

CH20 : +0.1585

TUE : -0.1234

FCVC : +0.0984

Age : +0.0707

MTRANS : -0.0607

=====

MÉTRICAS LDA – Conjunto de Prueba

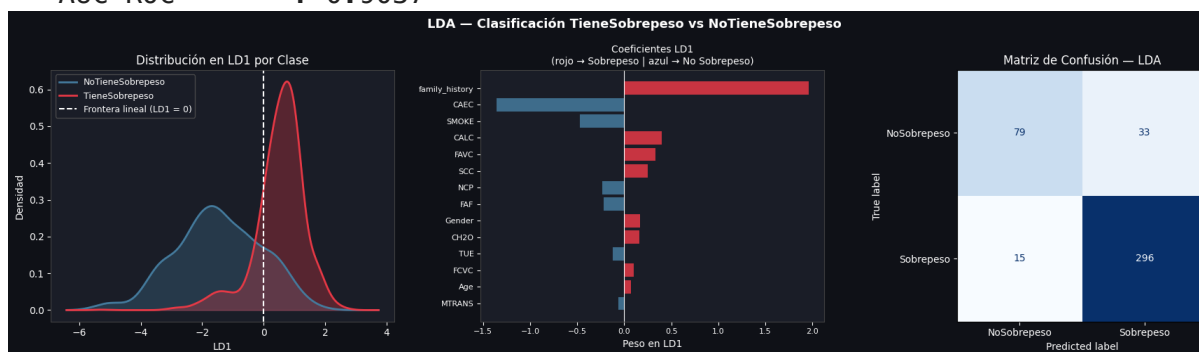
=====

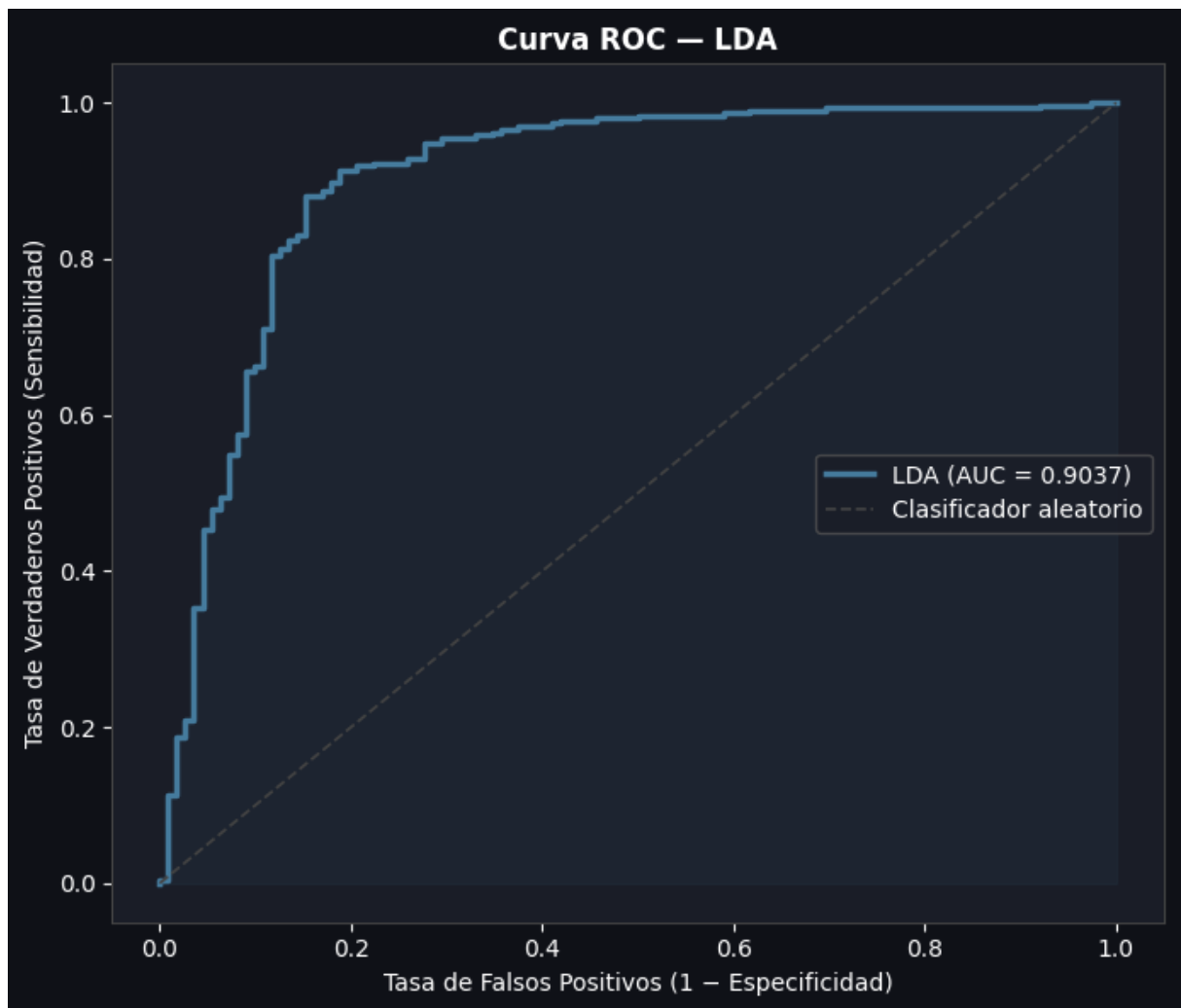
Accuracy : 88.65%

Sensibilidad : 95.18%

Especificidad : 70.54%

AUC-ROC : 0.9037





Para los dos modelos, LDA y Árboles de Decisión, se utiliza stratify para mantener el balance de clases, donde un total de 1552 registros (73.5%) se clasifican como TieneSobrepeso y 559 (26.5%) como NoTieneSobrepeso. En las pruebas Train y Test ambos tienen el mismo balance de 73.5%, lo que indica que no existe desbalance de clases y el modelo es apto para realizar la clasificación.

Obtenemos los coeficientes ordenados por magnitud que nos indican qué variables tienen mayor peso (positiva o negativamente) a la hora de tener sobrepeso. La variable más influyente es "family_history", lo que al igual que en anteriores reportes, puede significar que la influencia familiar y los hábitos de las personas que rodean a alguien en su familia influyen en su nivel de peso. Otra variable con un peso bastante importante fue "CAEC", que obtiene información sobre si la persona consume snacks entre comidas; su signo negativo indica que un mayor consumo entre comidas se asocia curiosamente a la clase NoTieneSobrepeso, lo cual podría explicarse porque esta variable también captura patrones de personas con peso insuficiente o normal que comen con más frecuencia pero en menor cantidad. Las variables SMOKE, CALC y FAVC son las siguientes con un coeficiente suficientemente relevante, pero muy alejadas de las dos principales. Observando la gráfica de distribución en LD1, la línea

discriminante logra una separación razonable entre las dos clases, sin embargo no es perfecta. Las distribuciones de TieneSobrepeso y NoTieneSobrepeso se traslapa en la zona central, lo que significa que hay un grupo de personas que el modelo no logra clasificar con confianza. Hacia los extremos la separación es más clara, pero en el medio la frontera lineal no es suficiente para distinguir bien entre ambas clases. Esto se refleja directamente en la Especificidad de 70.54%, que es la métrica más baja del modelo y confirma que esa zona de traslape afecta principalmente la identificación de personas sin sobrepeso. Con respecto a las métricas, el modelo obtiene un Accuracy de 88.65%, lo que indica que clasifica correctamente casi 9 de cada 10 casos. La Sensibilidad de 95.18% es especialmente relevante en este contexto: el modelo detecta correctamente el 95% de las personas que realmente tienen sobrepeso, lo cual es deseable en un problema de salud donde es más costoso no identificar un caso positivo. Finalmente, el AUC-ROC de 0.9037 confirma que el modelo tiene un buen desempeño global considerando todos los umbrales posibles.

Modelo basado en Árboles de Decisión

```
In [24]: # =====
# A2.2 – Árbol de Decisión
# Dataset: Obesidad (hábitos)
# Variable objetivo: TieneSobrepeso (1) / NoTieneSobrepeso (0)
# =====

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import networkx as nx
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OrdinalEncoder, LabelEncoder
from sklearn.metrics import (confusion_matrix, accuracy_score,
                             roc_curve, roc_auc_score)

import warnings
warnings.filterwarnings('ignore')

# -----
# 1. PREPROCESAMIENTO
# -----

df = pd.read_csv('Datos.csv')

sobrepeso_clases = ['Overweight_Level_I', 'Overweight_Level_II',
                   'Obesity_Type_I', 'Obesity_Type_II', 'Obesity_Type_III']
df['TieneSobrepeso'] = df['Obesity'].isin(sobrepeso_clases).astype(int)
df = df.drop(columns=['Height', 'Weight', 'Obesity'])
```

```

oe = OrdinalEncoder(
    categories=[['no', 'Sometimes', 'Frequently', 'Always'],
                ['no', 'Sometimes', 'Frequently', 'Always'],
                ['Walking', 'Bike', 'Motorbike', 'Public_Transportation']],
    handle_unknown='use_encoded_value', unknown_value=-1)
df[['CAEC', 'CALC', 'MTRANS']] = oe.fit_transform(df[['CAEC', 'CALC', 'MTRANS']])

le = LabelEncoder()
for col in ['Gender', 'family_history', 'FAVC', 'SMOKE', 'SCC']:
    df[col] = le.fit_transform(df[col])

feature_cols = [c for c in df.columns if c != 'TieneSobrepeso']
X = df[feature_cols]
y = df['TieneSobrepeso']

# -----
# 2. PARTICIÓN TRAIN / TEST (80/20, estratificado)
# -----

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# -----
# 3. ÁRBOL SIN PODA
# -----

dt_full = DecisionTreeClassifier(criterion='gini', random_state=42)
dt_full.fit(X_train, y_train)

acc_full_train = accuracy_score(y_train, dt_full.predict(X_train))
acc_full_test = accuracy_score(y_test, dt_full.predict(X_test))

print(f"Sin poda - Profundidad: {dt_full.get_depth()} | Hojas: {dt_full.get_n_leaves()}")
print(f"Accuracy train: {acc_full_train*100:.2f}% | test: {acc_full_test*100:.2f}%")

# -----
# 4. PODA - Cost Complexity Pruning
# -----

path = dt_full.cost_complexity_pruning_path(X_train, y_train)
ccp_alphas = path.ccp_alphas[:-1]

train_scores, test_scores = [], []
for alpha in ccp_alphas:
    dt = DecisionTreeClassifier(criterion='gini', ccp_alpha=alpha, random_state=42)
    dt.fit(X_train, y_train)
    train_scores.append(accuracy_score(y_train, dt.predict(X_train)))
    test_scores.append(accuracy_score(y_test, dt.predict(X_test)))

best_idx = np.argmax(test_scores)
best_alpha = ccp_alphas[best_idx]

```

```

dt_pruned = DecisionTreeClassifier(criterion='gini', ccp_alpha=best_alpha)
dt_pruned.fit(X_train, y_train)

acc_pruned_test = accuracy_score(y_test, dt_pruned.predict(X_test))

print(f"\nPodado - Profundidad: {dt_pruned.get_depth()} | Hojas: {dt_pruned.get_n_leaves()}")
print(f" Alpha óptimo: {best_alpha:.5f} | Accuracy test: {acc_pruned_test:.2f}")

# -----
# 5. MÉTRICAS DEL ÁRBOL PODADO
# -----

y_pred = dt_pruned.predict(X_test)
y_prob = dt_pruned.predict_proba(X_test)[:, 1]

cm = confusion_matrix(y_test, y_pred)
tn, fp, fn, tp = cm.ravel()
sensibilidad = tp / (tp + fn)
especificidad = tn / (tn + fp)
auc = roc_auc_score(y_test, y_prob)

print(f"\nMétricas árbol podado (test):")
print(f" Accuracy : {acc_pruned_test*100:.2f}%")
print(f" Sensibilidad : {sensibilidad*100:.2f}%")
print(f" Especificidad : {especificidad*100:.2f}%")
print(f" AUC-ROC : {auc:.4f}")

# -----
# FUNCIÓN: dibuja un árbol como grafo de nodos circulares
# -----

def dibujar_arbol(tree_clf, titulo, max_depth=5):
    """
    Visualiza el árbol como red de nodos circulares.
    - Azul → nodo con mayoría NoSobrepeso
    - Rojo → nodo con mayoría TieneSobrepeso
    - Tamaño del nodo proporcional al número de muestras
    - Opacidad proporcional a la pureza del nodo
    """
    tree_ = tree_clf.tree_
    BG = '#0D1117'
    C0 = '#457B9D' # NoSobrepeso
    C1 = '#E63946' # TieneSobrepeso
    EDGE = '#3D444D'

    G = nx.DiGraph()
    pos = {}
    node_colors = []
    node_sizes = []
    node_alphas = []
    node_list = []

```

```

# BFS para construir posiciones en layout de árbol
queue = [(0, None, 0, 0.5, 1.0)]
while queue:
    node, parent, depth, x, width = queue.pop(0)
    if depth > max_depth:
        continue
    G.add_node(node)
    pos[node] = (x, -depth)
    node_list.append(node)

    if parent is not None:
        G.add_edge(parent, node)

    vals = tree_.value[node][0]
    majority = int(np.argmax(vals))
    total_n = vals.sum()
    purity = vals[majority] / total_n

    node_colors.append(C1 if majority == 1 else C0)
    node_sizes.append(60 + 350 * (total_n / tree_.n_node_samples[0]
    node_alphas.append(0.35 + 0.65 * purity)

    left = tree_.children_left[node]
    right = tree_.children_right[node]
    half = width / 2
    if left != -1: queue.append((left, node, depth + 1, x - half
    if right != -1: queue.append((right, node, depth + 1, x + half

fig, ax = plt.subplots(figsize=(16, 6))
fig.patch.set_facecolor(BG)
ax.set_facecolor(BG)

nx.draw_networkx_edges(G, pos, ax=ax, edge_color=EDGE,
                        width=0.7, arrows=False, node_size=0)

for i, node in enumerate(node_list):
    nx.draw_networkx_nodes(G, pos, nodelist=[node], ax=ax,
                           node_color=[node_colors[i]],
                           node_size=[node_sizes[i]],
                           alpha=node_alphas[i],
                           linewidths=0)

# Leyenda
leyenda = [
    mpatches.Patch(color=C0, label='NoSobrepeso'),
    mpatches.Patch(color=C1, label='TieneSobrepeso'),
    mpatches.Patch(color='#888', alpha=0.4, label='Más transparente'),
    mpatches.Patch(color='#888', label='Más grande = más muestras')
]
ax.legend(handles=leyenda, loc='lower right',
          facecolor='#161B22', edgecolor='#30363D',

```

```

        labelcolor='white', fontsize=9, framealpha=0.9)

    ax.set_title(titulo, color='white', fontsize=12, fontweight='bold')
    ax.axis('off')
    plt.tight_layout()
    plt.show()

# -----
# GRÁFICA 1: Árbol sin poda (primeros 5 niveles)
# -----

dibujar_arbol(
    dt_full,
    titulo=(f'Árbol Sin Poda · Profundidad: {dt_full.get_depth()} · '
            f'Hojas: {dt_full.get_n_leaves()} · '
            f'Accuracy test: {acc_full_test*100:.1f}%'
            f'\n(Visualizando todos los niveles del árbol sin podar ({
    max_depth=18
)

# -----
# GRÁFICA 2: Árbol podado (primeros 5 niveles)
# -----

dibujar_arbol(
    dt_pruned,
    titulo=(f'Árbol Podado ·  $\alpha$  = {best_alpha:.5f} · '
            f'Profundidad: {dt_pruned.get_depth()} · '
            f'Hojas: {dt_pruned.get_n_leaves()} · '
            f'Accuracy test: {acc_pruned_test*100:.1f}%'
            f'\n(Visualizando todos los niveles del árbol podado ({dt_
    max_depth=18
)

# -----
# GRÁFICA 3: Curva de poda – muestra el efecto del alpha
# -----

fig, ax = plt.subplots(figsize=(10, 5))
fig.patch.set_facecolor('#0D1117')
ax.set_facecolor('#161B22')

ax.plot(ccp_alphas, [s * 100 for s in train_scores],
        color='#457B9D', lw=2, label='Train')
ax.plot(ccp_alphas, [s * 100 for s in test_scores],
        color='#E63946', lw=2, label='Test')
ax.axvline(best_alpha, color='#E9C46A', linestyle='--', lw=1.5,
           label=f' $\alpha$  óptimo = {best_alpha:.5f}')

# Anotaciones de los dos extremos
ax.annotate(f'Sin poda\n{acc_full_test*100:.1f}%',

```

```

xy=(ccp_alphas[0], test_scores[0] * 100),
xytext=(best_alpha * 3, test_scores[0] * 100 - 2.5),
color='#aaa', fontsize=8,
arrowprops=dict(arrowstyle='->', color='#aaa', lw=1))
ax.annotate(f'Podado\n{acc_pruned_test*100:.1f}%',
xy=(best_alpha, acc_pruned_test * 100),
xytext=(best_alpha * 6, acc_pruned_test * 100 + 1),
color='#E9C46A', fontsize=8,
arrowprops=dict(arrowstyle='->', color='#E9C46A', lw=1))

ax.set_xlabel('Alpha ( $\alpha$ )', color='white', fontsize=10)
ax.set_ylabel('Accuracy (%)', color='white', fontsize=10)
ax.set_title('Curva de Poda – Efecto del Alpha sobre el Accuracy',
color='white', fontsize=12, fontweight='bold')
ax.tick_params(colors='white')
ax.spines[:].set_color('#30363D')
ax.legend(facecolor='#161B22', edgecolor='#30363D',
labelcolor='white', fontsize=9)

plt.tight_layout()
plt.show()

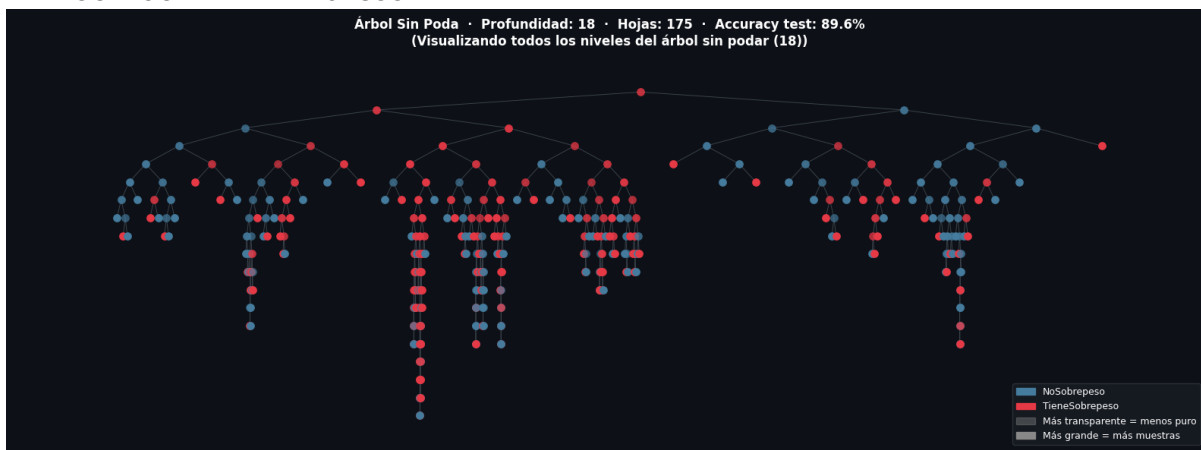
```

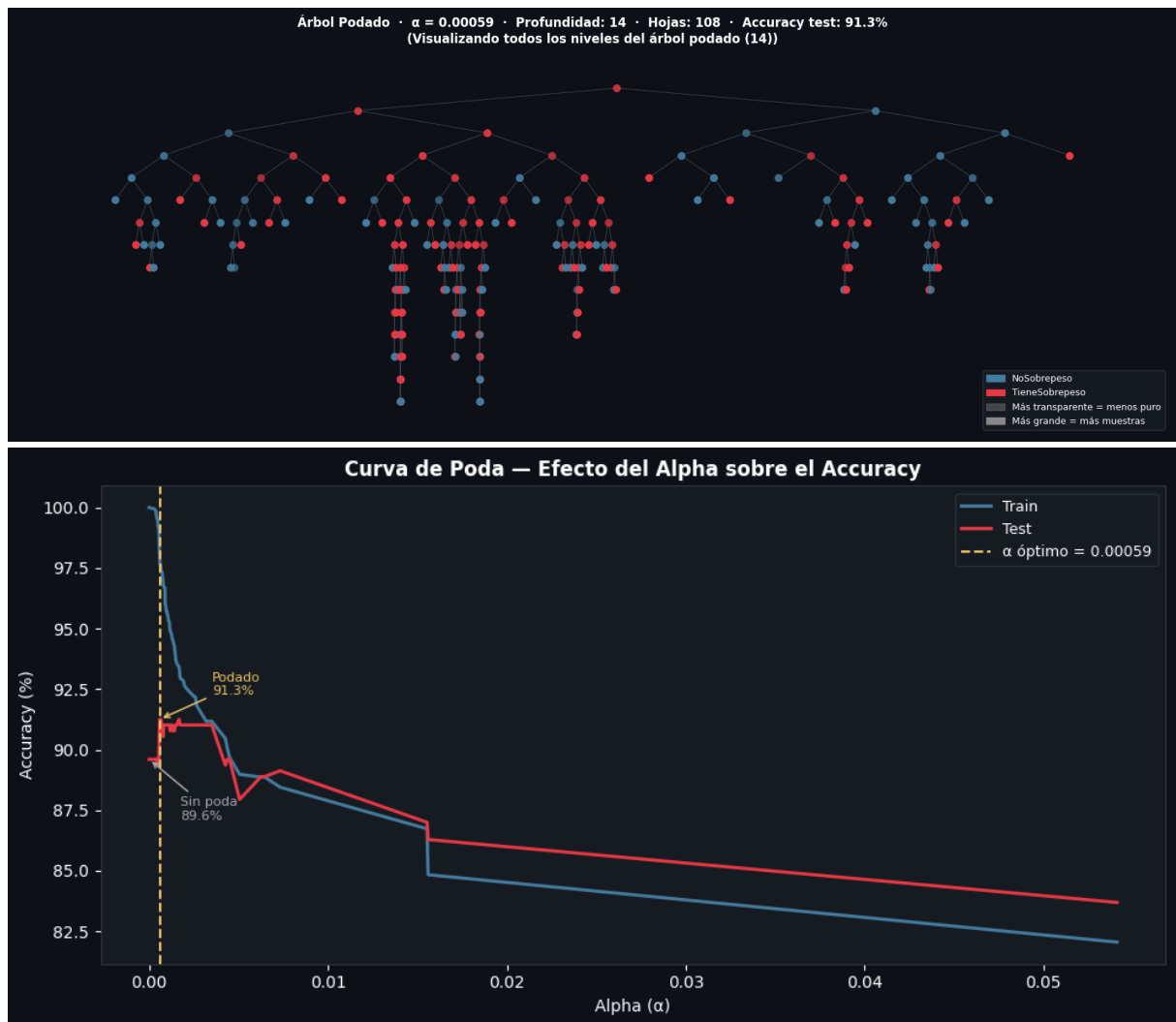
Sin poda – Profundidad: 18 | Hojas: 175
Accuracy train: 100.00% | test: 89.60%

Podado – Profundidad: 14 | Hojas: 108
Alpha óptimo: 0.00059 | Accuracy test: 91.25%

Métricas árbol podado (test):

Accuracy : 91.25%
Sensibilidad : 92.60%
Especificidad : 87.50%
AUC-ROC : 0.8994





Para nuestro modelo, proporcionamos dos gráficas comparativas que representan lo que hace nuestro modelo, va dividiendo los datos en renglones a base de diferentes preguntas como "¿Hay antecedentes familiares?" "¿La persona fuma?", y dependiendo de la respuesta, se crea una nueva hoja a la izquierda o a la derecha. La primera gráfica sirve para visualizar el árbol completo, tiene un total de 18 niveles, y 175 hojas. Con respecto a los resultados, observamos un Accuracy en entrenamiento de 100% y un test de 89.6%. Esto es una mala señal pues nos indica que hay un overfitting es decir, el modelo memorizó los datos completamente y no fue capaz de procesar correctamente los nuevos datos. Para obtener un mejor modelo, es necesario realizar una poda, donde se reduzcan las hojas para reducir el overfitting que hemos observado. Así se eliminan los subárboles que han crecido en exceso y no ayudan a la precisión esperada con nuevos datos. Para esto, se aplicó Cost Complexity Pruning, este método agrega una penalización por complejidad, donde cuanto más grande la penalización sea, más ramas se eliminarán. Para esto, el modelo prueba todas las penalizaciones posibles y se eligió un alpha de 0.00059, ya que dió el accuracy test más alto: 91.25%. Según las gráficas del árbol de decisión, la variable que

resultó ser la más importante fue "family_history", es decir, si la persona tiene antecedentes familiares de obesidad. Esta variable aparece en los primeros nodos del árbol, lo que significa que es la primera pregunta que el modelo hace para dividir los datos. Esto tiene sentido considerando que los hábitos alimenticios y el estilo de vida suelen ser compartidos dentro de una familia, lo que convierte a los antecedentes familiares en un predictor fuerte del sobrepeso. Los resultados finales que ofrece este modelo son positivos, con un buen Accuracy, una Sensibilidad de 92.6%, que indica que detecta correctamente el 92.6% de personas que realmente tienen sobrepeso. Una Especificidad un poco más baja de un 87.5% que indica que también tiene un porcentaje alto en detectar a quienes no tienen sobrepeso. Para visualizar el efecto del alpha sobre el modelo, se generó una curva de poda que muestra el accuracy en entrenamiento y en prueba para cada valor de alpha probado. Se puede observar que el accuracy de entrenamiento comienza en 100% y va disminuyendo conforme alpha aumenta, lo cual tiene sentido ya que a mayor penalización el árbol pierde capacidad de ajustarse a los datos. Por otro lado, el accuracy de prueba sube inicialmente, lo que indica que se está reduciendo el overfitting, hasta alcanzar su punto máximo en $\alpha = 0.00059$ con un 91.25%. A partir de ese punto, el accuracy de prueba también comienza a caer, señal de que el árbol ya está siendo demasiado simplificado y pierde capacidad predictiva. El alpha óptimo se encuentra justo en ese pico, donde se logra el mejor balance entre complejidad del modelo y capacidad de generalización.

Evaluación de ambos modelos

Para este proyecto, se considera que el método de Árboles de Decisión nos presenta un mejor resultado a la hora de la clasificación, obtiene mejores métricas de Accuracy y Especificidad bastante superior al modelo LDA, siendo esta última métrica la que tiene una diferencia más notable: 87.50% contra 70.54%, lo que indica que el modelo LDA se equivoca con más frecuencia al intentar identificar a personas que no tienen sobrepeso. Esto indica que este modelo es mejor descartando a personas que no tienen sobrepeso. La Sensibilidad es la única métrica en la que el modelo LDA supera a los Árboles de Decisión por un 3%. En cuanto la curva AUC-ROC, ambos modelos son muy parecidos teniendo un desempeño similar. En el modelo LDA, en la gráfica se puede observar como se traslapa en la zona del medio, lo que indica que tiene problemas para separar correctamente a las personas sin sobrepeso, que son las que caen en esa zona de confusión. En el caso del árbol, las gráficas mostraban claramente el problema de overfitting antes de la poda, donde el modelo había memorizado los datos de entrenamiento pero no funcionaba igual de bien con

datos nuevos. Tras aplicar la poda, ese problema se corrigió y el desempeño mejoró. El LDA necesita que los datos cumplan ciertas condiciones para funcionar bien, entre ellas que los datos de cada clase se distribuyan de forma similar a una campana y que ambas clases tengan una variabilidad parecida. En nuestro dataset eso no ocurre del todo, lo que puede explicar que el modelo tenga más dificultades en ciertos casos. El Árbol de Decisión no tiene ese tipo de restricciones, simplemente va dividiendo los datos con preguntas hasta encontrar patrones, lo que le da más libertad para adaptarse a la realidad de los datos. Por esto es que se considera que el método de Árboles de Decisión después de la poda es el mejor modelo para este reporte. Tiene mejor Accuracy, una Especificidad mucho más alta y no depende de condiciones que nuestros datos no cumplen del todo. El LDA sigue siendo un buen modelo, y de hecho detecta un poco mejor los casos positivos, pero su debilidad para identificar correctamente a personas sin sobrepeso lo hace menos confiable en general. Si el objetivo fuera únicamente no dejar pasar ningún caso de sobrepeso sin importar cuántos errores se cometen en el otro lado, el LDA podría ser una opción, pero para una clasificación balanceada y confiable, el árbol gana.

Referencias

Anthropic. (2025). Claude (claude-sonnet-4-6) [Modelo de lenguaje de inteligencia artificial]. <https://www.claude.ai>

Martínez Torteya, A. (2024). Árboles de Decisión [Presentación de clase]. Universidad de Monterrey.

Martínez Torteya, A. (2024). Calidad del Modelo y Validación Cruzada [Presentación de clase]. Universidad de Monterrey.

Martínez Torteya, A. (2024). Clasificación – Regresión Logística [Presentación de clase]. Universidad de Monterrey.

Martínez Torteya, A. (2024). Linear Discriminant Analysis [Presentación de clase]. Universidad de Monterrey.